

Table of Contents

1. Introduction	1
1.1 Project Aims and Motivation	1
2. Dataset Description and Exploratory Data Analysis (EDA)	1
2.1 Dataset Description	1
2.2 Exploratory Data Analysis (EDA)	5
3. Data Preprocessing	10
3.1 Null / Missing Values	10
3.2 Removing Outliers	11
3.3 Dataset Splitting	12
3.4 Encoding and Feature Scaling	12
3.5 Class Imbalance Handling	14
5. Model Training & Testing	15
5.1 Supervised Learning	15
5.1.1 Logistic Regression	15
5.1.2 Decision Tree	16
5.1.3 Neural Network	17
5.1.4 K Nearest Neighbour	18
5.2 Unsupervised Learning	18
5.2.1 K Means	18
6. Comparative Analysis & Model Selection	21
6.1 Comparison of Accuracy	21
6.2 Precision, Recall, and Confusion Matrices	21
6.2.1 Precision	22
6.2.2 Recall	22
6.2.3 Confusion Matrices	22
6.3 AUC Score and ROC Curve	25
6.4 Model Selection	26
7. Conclusion	26

1. Introduction

In this project, we implemented various machine learning models on a given [dataset](#). This includes, In depth analysis of the data, Preprocessing of the dataset to best match the models, and finally a comparative analysis of all the models on their performance. The whole python notebook can be found in this [colab file](#).

1.1 Project Aims and Motivation

This project addresses the critical challenge of predicting timely delivery in the e-commerce industry by applying machine learning algorithms. Using the [E-commerce Shipping Dataset](#), which is composed of various logistical and customer-related features, the project's primary goal is to develop and compare various predictive models capable of classifying shipments as either on-time or delayed. This binary classification problem has significant business benefits, as early identification of shipments having the risk of being delayed can enable proactive logistical interventions to enhance customer satisfaction.

The objective is to perform data analysis, explore preprocessing techniques, apply supervised learning models such as Logistic Regression, Decision Tree, Neural Network, and K-Nearest Neighbors to compare their performance and determine the most effective approach for this task. The insights gained from this project can help businesses optimize their supply chains and improve delivery reliability, resulting in higher customer satisfaction.

2. Dataset Description and Exploratory Data Analysis (EDA)

Before applying any machine learning models, we must analyze the dataset to understand its nature and type. From the dataset, we can understand what type of problem we are going to solve. In this section, we will tackle the Descriptive and EDA part of the project.

2.1 Dataset Description

- **Features and Datapoints:** The dataset contains 10999 datapoints and 12 features including both input features (e.g., 'Warehouse_block', 'Mode_of_Shipment', 'Product_importance', 'Gender', 'Customer_care_calls', 'Customer_rating', 'Cost_of_the_Product', 'Prior_purchases', 'Discount_offered', 'Weight_in_gms') and the target variable ('Reached.on.Time_Y.N'). But the feature 'ID' is irrelevant to our project so we will drop this feature before moving forward.

```
print("Shape of dataset is: ", df.shape)
print(f"Dataset contain {df.shape[0]} rows & {df.shape[1]} columns")

Shape of dataset is: (10999, 11)
Dataset contain 10999 rows & 11 columns
```

```
df = pd.read_csv('E-commerce Shipping Dataset.csv')
print("First 10 rows of the Dataset")
target = "Reached.on.Time_Y.N"
df.head(10)
```

First 10 rows of the Dataset

	ID	Warehouse_block	Mode_of_Shipment	Customer_care_calls	Customer_rating	Cost_of_the_Product	Prior_purchases	Product_importance	Gender	Discount_offered	Weight_in_gms	Reached.on.Time_Y.N
0	1	D	Flight	4	2	177	3	low	F	44	1233	1
1	2	F	Flight	4	5	216	2	low	M	59	3088	1
2	3	A	Flight	2	2	183	4	low	M	48	3374	1
3	4	B	Flight	3	3	176	4	medium	M	10	1177	1
4	5	C	Flight	2	2	184	3	medium	F	46	2484	1
5	6	F	Flight	3	1	162	3	medium	F	12	1417	1
6	7	D	Flight	3	4	250	3	low	F	3	2371	1
7	8	F	Flight	4	1	233	2	low	F	48	2804	1
8	9	A	Flight	3	4	150	3	low	F	11	1861	1
9	10	B	Flight	3	2	164	3	medium	F	29	1187	1

- **Problem Type:** This is a classification problem because the objective is to predict a discrete, categorical outcome (on-time or delayed) rather than a continuous numerical value. The target variable, 'Reached.on.Time_Y.N', has two discrete classes representing whether a shipment reached its destination on time (1) or not (0). The goal is to predict one of these predefined categories, based on the other provided features, rather than predicting a continuous value.
- **Feature Types:** The dataset includes both quantitative (numerical) and categorical features. Quantitative features, such as Cost_of_the_Product and Weight_in_gms, are measurable attributes. Categorical features, such as Mode_of_Shipment, Warehouse_block, and Product_importance, represent distinct groups.

```
numerical_data = df.select_dtypes(include=np.number)

numerical_features=numerical_data.columns.tolist()
print(f'There are {len(numerical_features)} numerical features:', '\n')
print(numerical_features)
```

There are 7 numerical features:

```
['Customer_care_calls', 'Customer_rating', 'Cost_of_the_Product', 'Prior_purchases', 'Discount_offered', 'Weight_in_gms', 'Reached.on.Time_Y.N']
```

```
[ ] categorical_data = df.select_dtypes(include='object')

categorical_features = categorical_data.columns.tolist()

print(f'There are {len(categorical_features)} categorical features:', '\n')
print(categorical_features)

There are 4 categorical features:

['Warehouse_block', 'Mode_of_Shipment', 'Product_importance', 'Gender']
```

- **Descriptive Analysis:** For both categorical and quantitative features in the dataset, we got the descriptive analysis result for them to understand their characteristics.

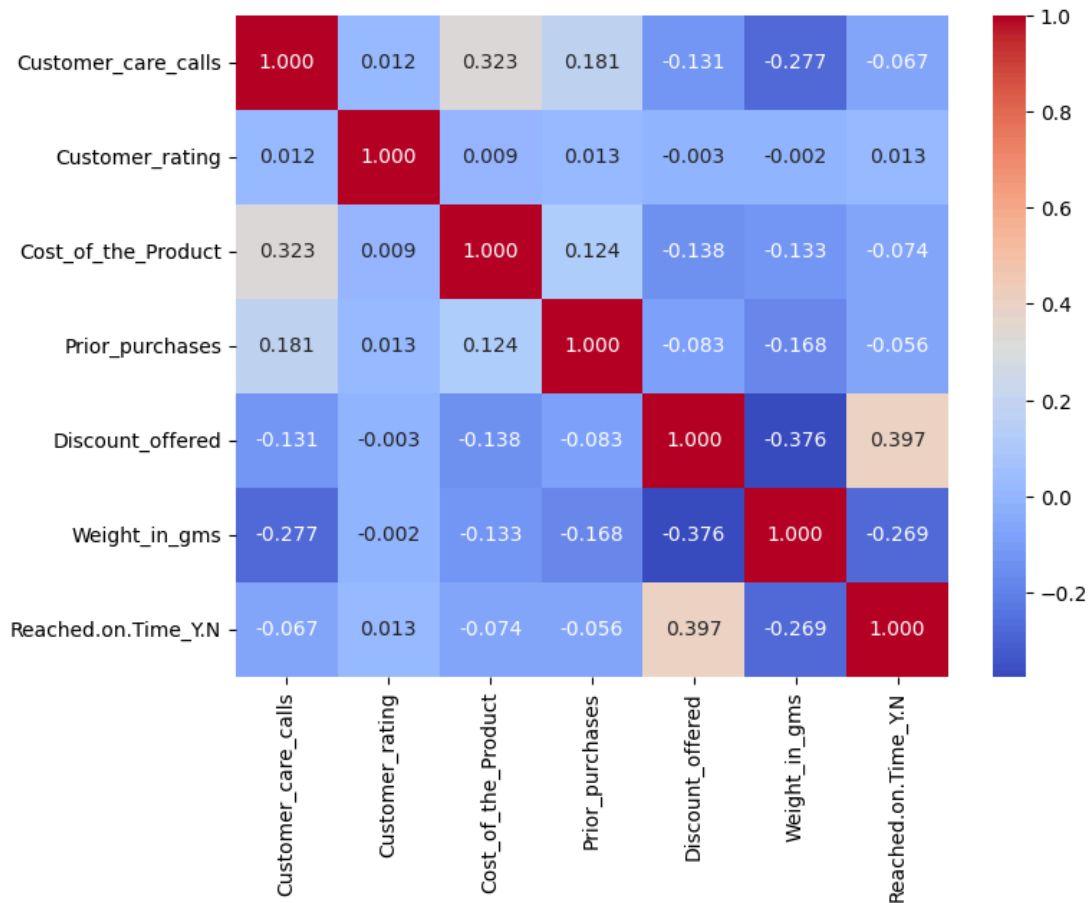
	Customer_care_calls	Customer_rating	Cost_of_the_Product	Prior_purchases	Discount_offered	Weight_in_gms	Reached.on.Time_Y.N
count	10999.000000	10999.000000	10999.000000	10999.000000	10999.000000	10999.000000	10999.000000
mean	4.054459	2.990545	210.196836	3.567597	13.373216	3634.016729	0.596691
std	1.141490	1.413603	48.063272	1.522860	16.205527	1635.377251	0.490584
min	2.000000	1.000000	96.000000	2.000000	1.000000	1001.000000	0.000000
25%	3.000000	2.000000	169.000000	3.000000	4.000000	1839.500000	0.000000
50%	4.000000	3.000000	214.000000	3.000000	7.000000	4149.000000	1.000000
75%	5.000000	4.000000	251.000000	4.000000	10.000000	5050.000000	1.000000
max	7.000000	5.000000	310.000000	10.000000	65.000000	7846.000000	1.000000

The numerical features of the dataset shows their mean, standard deviation, min, max and the spread of values at 25%, 50%, 75%, and max. We can get an idea about the scale and distribution of numerical values for these features based on the information.

	Warehouse_block	Mode_of_Shipment	Product_importance	Gender
count	10999	10999	10999	10999
unique	5	3	3	2
top	F	Ship	low	F
freq	3666	7462	5297	5545

The categorical features of the dataset is represented with their unique values, and most repeated values. From this we can understand how distributed the categorical features are and gives us an idea about how to encode them in the preprocessing step.

- **Correlation:** A test was performed using a heatmap on the numerical features. The test revealed there is no significant correlation between the features, and we can also observe a low-moderate correlation between the feature Discount_offered and the target column 'Reached.on.Time_Y.N'. The feature 'Customer_care_calls' and 'Cost_of_the_product' show weak positive correlation.



- Imbalance_dataset:** Class imbalance may affect a model's learning when some categories in a dataset are noticeably underrepresented in comparison to others (for the target feature). In the E-commerce Shipping Dataset, the class distribution for the target feature 'Reached.on.Time_Y.N' is as follows:

1 (On-time): 6563 (approximately 59.7%)

0 (Delayed): 4436 (approximately 40.3%)

```

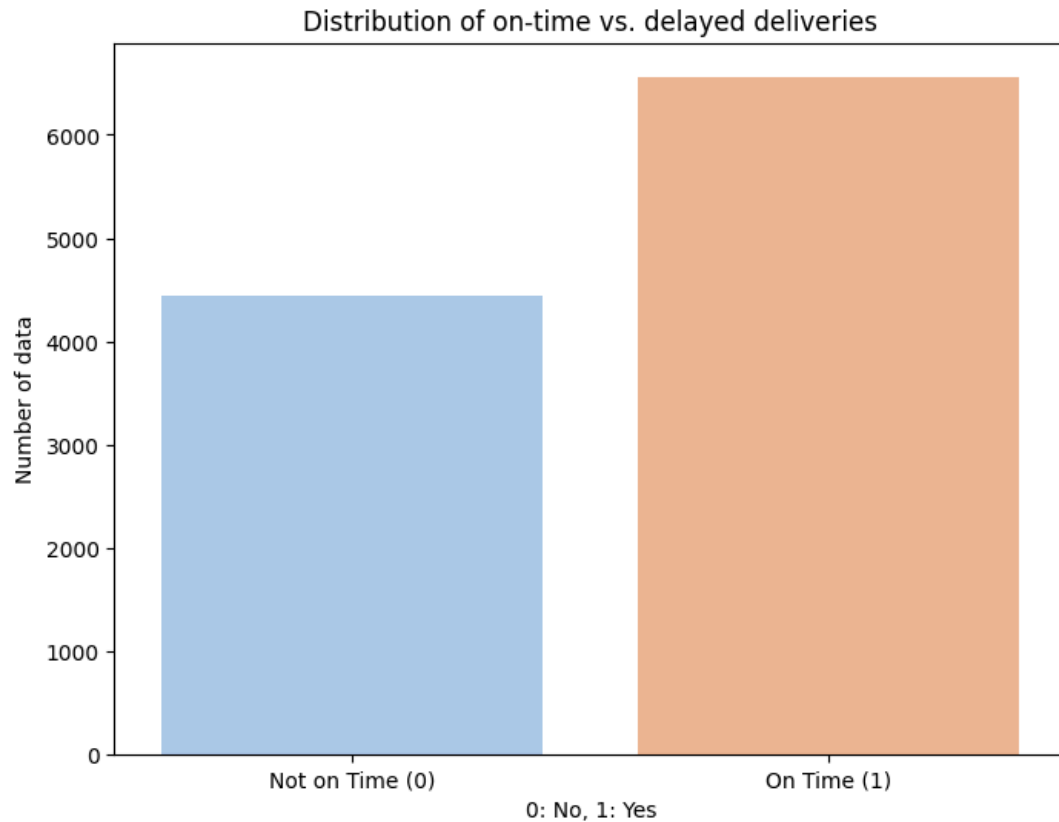
Imbalanced Dataset

class_counts = df[target].value_counts()
print("Class Distribution:")
print(class_counts)

plt.figure(figsize=(8, 6))
sns.barplot(x=class_counts.index, y=class_counts.values, palette='pastel')
plt.title('Distribution of on-time vs. delayed deliveries')
plt.xlabel('0: No, 1: Yes')
plt.ylabel('Number of data')
plt.xticks(ticks=[0,1], labels=['Not on Time (0)', 'On Time (1)'])
plt.show()

Class Distribution:
Reached.on.Time_Y.N
1    6563
0    4436
Name: count, dtype: int64
/tmp/ipython-input-2669527817.py:6: FutureWarning:

```



2.2 Exploratory Data Analysis (EDA)

- **Numerical Data Variance:** From the variance of the numerical features we can understand the spread and scale of the data. A high variance value of 'Weight_in_gms' means the values are widely spread out. On the other hand, a low variance value of 'Customer_care_calls' means the values of this feature are tightly clustered.

Customer_care_calls	1.302999e+00
----------------------------	--------------

Customer_rating	1.998274e+00
------------------------	--------------

Cost_of_the_Product	2.310078e+03
----------------------------	--------------

Prior_purchases	2.319103e+00
------------------------	--------------

Discount_offered	2.626191e+02
-------------------------	--------------

Weight_in_gms	2.674459e+06
----------------------	--------------

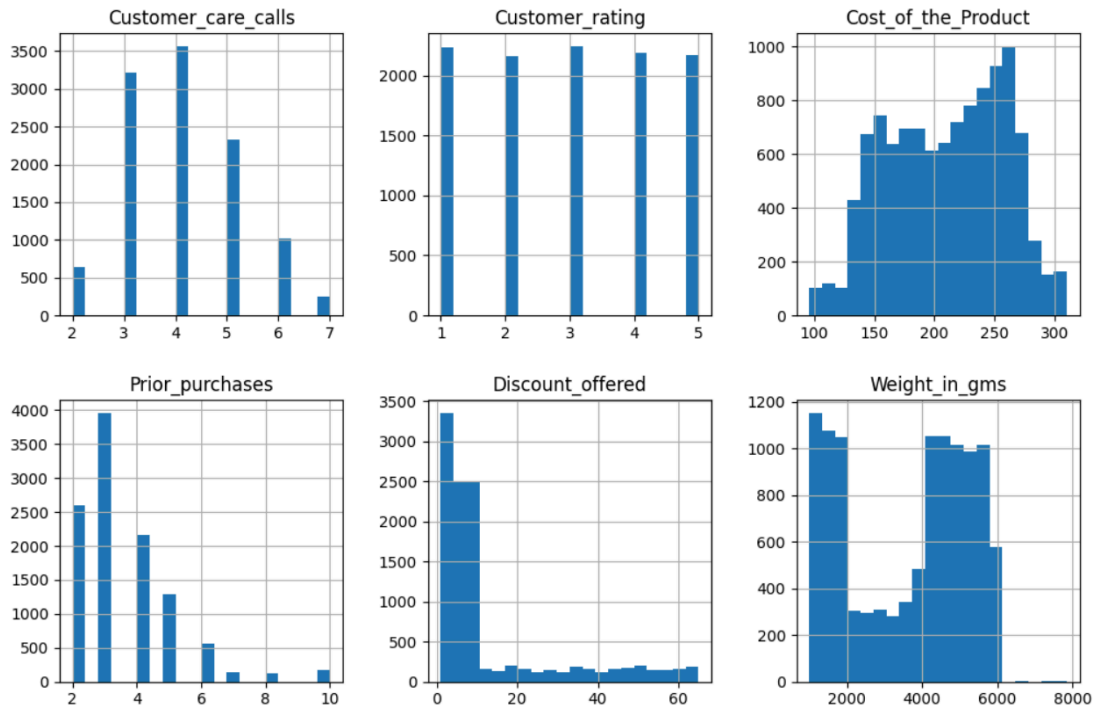
Reached.on.Time_Y.N	2.406728e-01
----------------------------	--------------

- **Numerical Data Skew:** We can achieve the understanding of the asymmetric distribution of the numerical features based on their skew value. Higher skew indicates the uneven distribution of the data, often due to outliers on either one side.

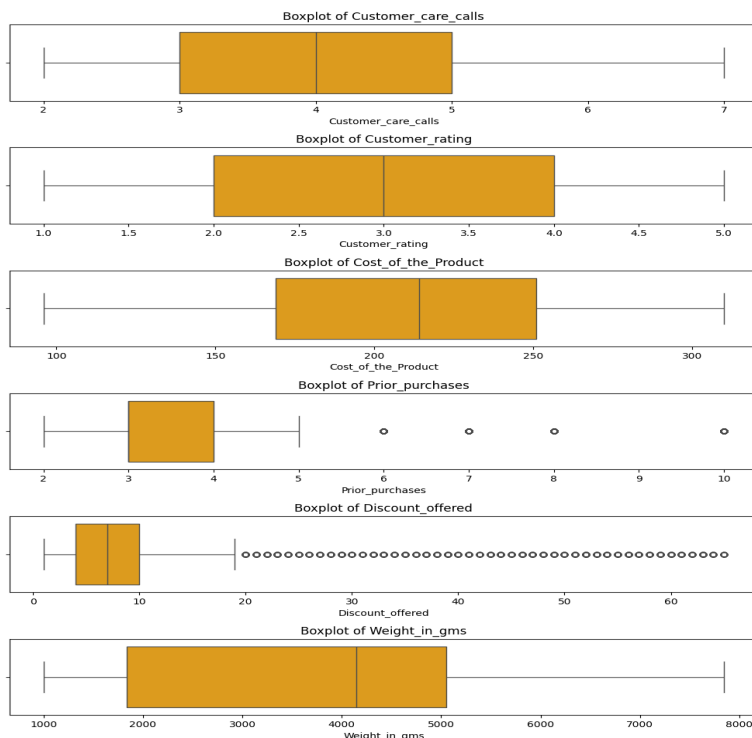
Customer_care_calls	0.391926
Customer_rating	0.004360
Cost_of_the_Product	-0.157117
Prior_purchases	1.681897
Discount_offered	1.798929
Weight_in_gms	-0.249747
Reached.on.Time_Y.N	-0.394257

Interpretation:

- Customer_care_calls (0.3919) = Slightly Right Skewed
 - Customer_rating (0.0044) = Almost symmetrical
 - Cost_of_the_product (-0.1571) = Slightly left Skewed
 - Prior_purchase (1.682) = Highly Right Skewed. Most people have less than 4 purchases, some has upto 10.
 - Discount_offered (1.7989) = Highly Right Skewed. Most orders have 10% discount, some goes up to 65%.
 - Weight_in_gms (-0.2497) = Slightly left skewed.
 - Reached.on.Time.Y.N (-0.3943) = Slightly left skewed. More orders end up reaching on time.
- **Histogram:** We represent the frequency distribution of the numerical features by grouping data into bins and displaying the count of observations in each bin. We can understand the underlying pattern of a feature by looking at the histogram. Central tendency, Spread, symmetry (bell curve) or skewed, etc. can be derived and understood from histograms of numerical features.

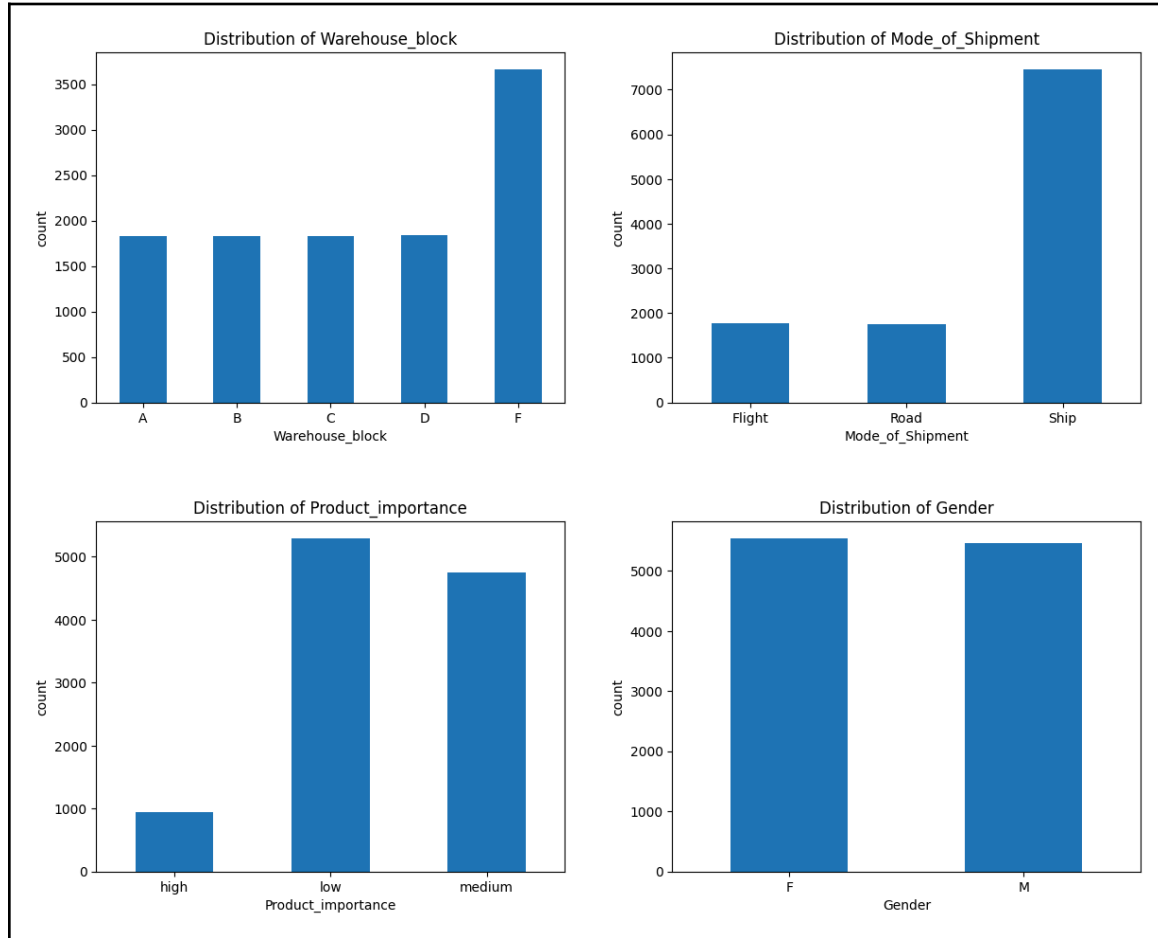


- Box plot to identify Outliers:** Outliers can have unwanted impact on machine learning models such as, higher outliers values might trick a model into thinking they are more important than they actually are for the decision making. The box plot graph of the numerical features will represent the amount and spread of outliers for each one.



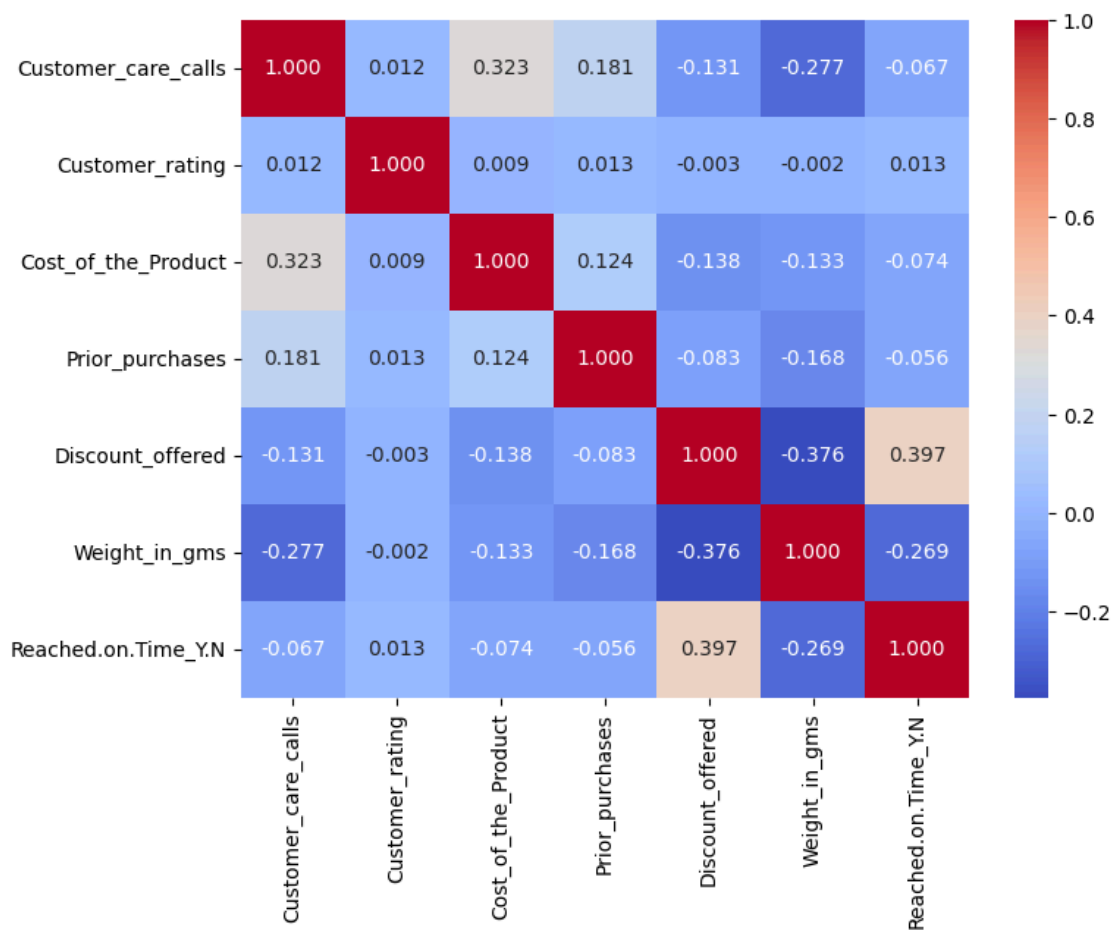
‘Prior_purchases’ and ‘Discount_offered’ features have noticeable outliers.

- **Unique Values:** The number of unique values in numerical features and categorical features represents the nature of the dataset. Although it is more significant for the categorical feature and this information is crucial for encoding the data for the models. We can have an understanding of the distribution of categorical features among it's unique values by graphing a Bar plot.



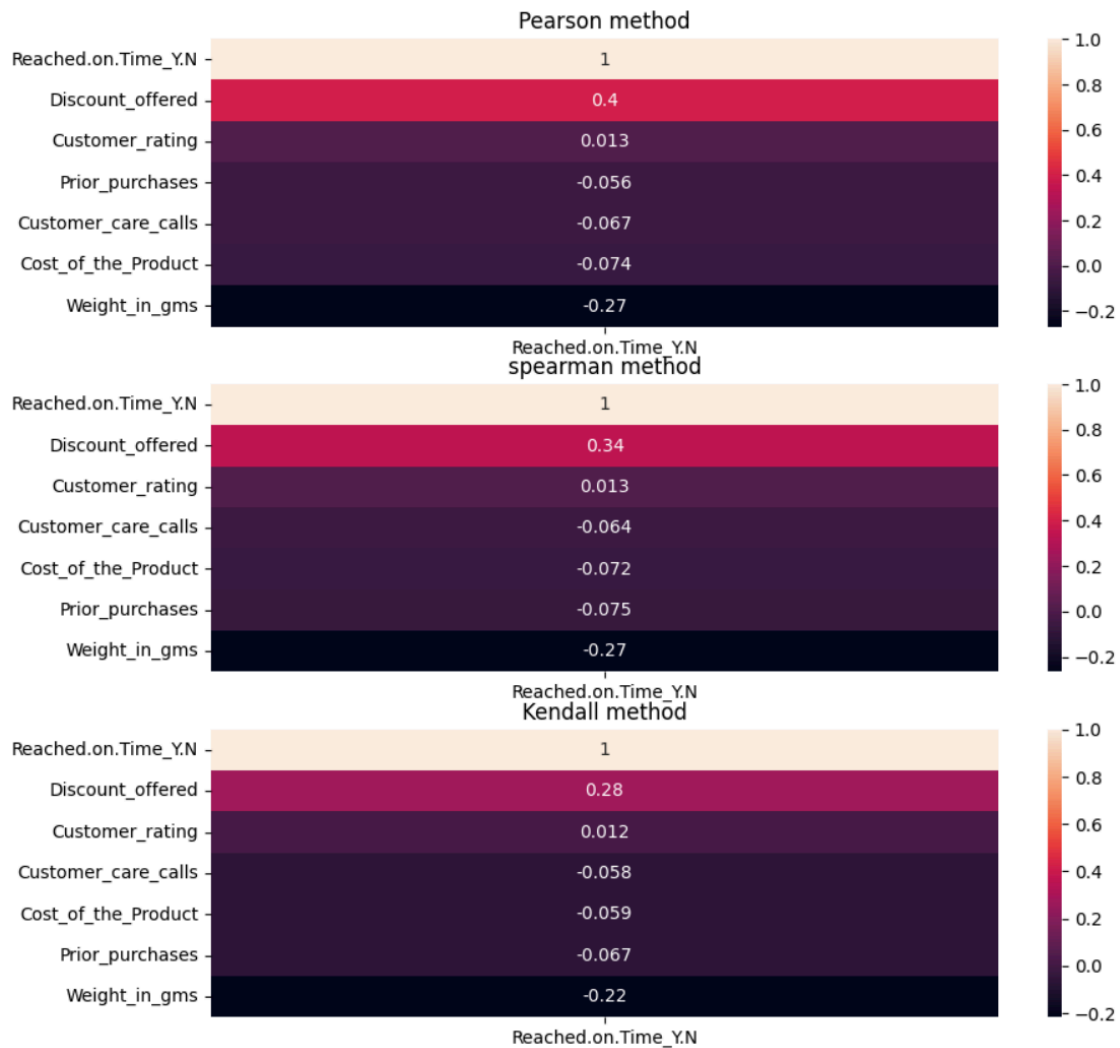
- **Feature Correlation Analysis:** The strength and direction of the linear relationship between each pair of numerical features can be visualized using a correlation heatmap. The values that have strongest correlation with our target feature ‘Reached.on.Time_Y.N’ are likely to have most impact on the predictions of our models. The correlation between features other than the target feature can give us the idea which two features are implying the same outcome so we can keep one and drop another to make a balanced prediction.

Correlation (r)	Interpretation
+1.0	Perfect positive correlation
0.7 - 0.9	Strong positive correlation
0.4 - 0.6	Moderate positive correlation
0.1 - 0.3	Weak positive correlation
0.0	No correlation
-0.1 - -0.3	Weak negative correlation
-0.4 - -0.6	Moderate negative correlation
-0.7 - -0.9	Strong negative correlation
-1.0	Perfect negative correlation



Comment: 'Discount_offered' and Target feature have moderate positive correlation, 'Discount_offered' and 'Weight_in_gms' have moderate negative correlation. 'Cost_of_the_product' and 'Customer_care_calls' have weak positive correlation.

As the correlation between target values and other features hold significant importance for the models outcome, we will represent the data by adding other correlation calculation methods.



3. Data Preprocessing

Data preprocessing is a crucial step in preparing the dataset for machine learning models. The following sections detail the identified faults in the raw data and the solutions applied to address them.

3.1 Null / Missing Values

An initial analysis of the Dataset revealed that there were no null values present in any of the features. This is an ideal scenario as it eliminates the need for data imputation or dropping incomplete rows, which can be a complex and time-consuming part of the data preprocessing

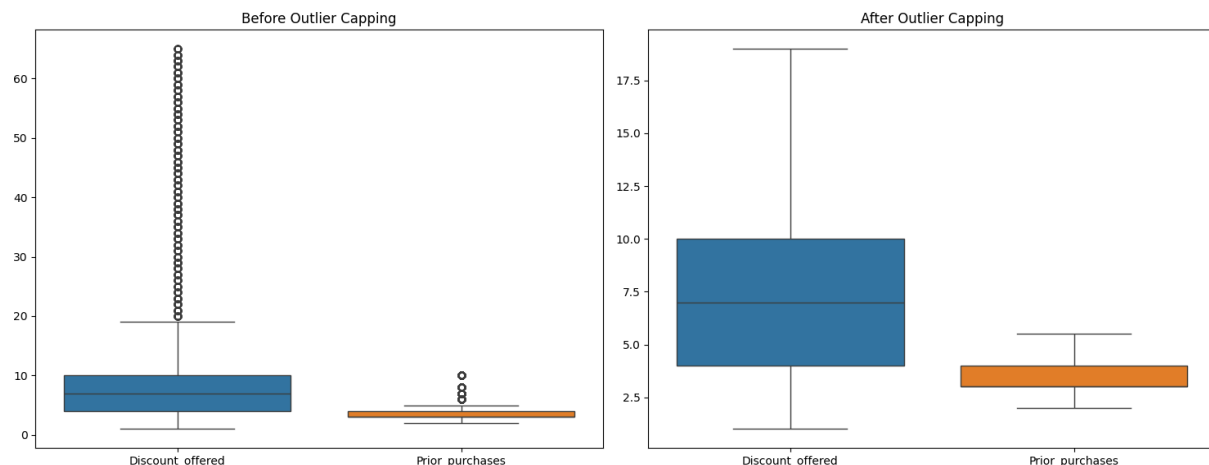
pipeline. The absence of missing data ensures that the entire dataset can be used for training and testing the machine learning models, leading to more robust and reliable predictions.

1 numerical_data.isnull().sum()		1 categorical_data.isnull().sum()	
	0		0
Customer_care_calls	0	Warehouse_block	0
Customer_rating	0	Mode_of_Shipment	0
Cost_of_the_Product	0	Product_importance	0
Prior_purchases	0	Gender	0
Discount_offered	0		
Weight_in_gms	0		
Reached.on.Time_Y.N	0		

3.2 Removing Outliers

When we analyzed the dataset using Boxplot, we clearly saw that two features had outliers which range outside of maximum values. Outliers can skew the dataset distribution, distort scaling and increase variance which can have a negative effect on the model .

The Interquartile Range (IQR) is used to remove outliers because it is based on the middle 50% of the data (between Q1[25%] and Q3[75%]) and is not affected by extreme values. By defining lower fence using $(Q1 - 1.5 \times IQR)$ and upper fence using $(Q3 + 1.5 \times IQR)$, we can identify and remove outliers, making the data cleaner and more reliable for analysis or modeling.



3.3 Dataset Splitting

To ensure the models are evaluated on unseen data, the dataset was split into training and testing sets. This prevents the models from simply memorizing the training data. Given the imbalanced nature of the target variable, a stratified split was used. This ensures that the proportion of each class in the training set is the same as in the testing set, which is crucial for fair and accurate model evaluation.

```
from sklearn.model_selection import train_test_split

X = df.drop(target, axis=1)
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

print("Shape of X_train:", X_train.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_test:", y_test.shape)
```

```
Shape of X_train: (7699, 10)
Shape of y_train: (7699,)
Shape of X_test: (3300, 10)
Shape of y_test: (3300,)
```

3.4 Encoding and Feature Scaling

Categorical Values

- **Fault:** The dataset contains multiple categorical features (e.g., Warehouse_block, Mode_of_Shipment, Product_importance, Gender) that are not in a numerical format. Machine learning models require numerical inputs to perform calculations and learn from the data.
- **Solution:** The categorical features were transformed into numerical representations using Ordinal, and One-Hot Encoding. Ordinal encoding was used for features with a natural order like 'Product_importance' with [low, medium, high] importance order. On the other hand, One-Hot Encoding method creates new binary columns for each unique category (except 1), ensuring that the model can process the data without assuming any ordinal relationship between them.

```

from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.compose import ColumnTransformer

numerical_features_X = X.select_dtypes(include=np.number).columns.tolist()
one_hot_features = ['Warehouse_block', 'Mode_of_Shipment', 'Gender']
ordinal_features = ['Product_importance']

importance_order = ['low', 'medium', 'high']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features_X),
        ('cat_onehot', OneHotEncoder(handle_unknown='ignore'), one_hot_features),
        ('cat_ordinal', OrdinalEncoder(categories=[importance_order]), ordinal_features)
    ],
    remainder='passthrough'
)

X_train_processed = preprocessor.fit_transform(X_train)

X_test_processed = preprocessor.transform(X_test)

print("Preprocessing successful using the scikit-learn pipeline!")
print("Shape of processed training data:", X_train_processed.shape)
print("Shape of processed testing data:", X_test_processed.shape)

```

```

➡ Preprocessing successful using the scikit-learn pipeline!
Shape of processed training data: (7699, 17)
Shape of processed testing data: (3300, 17)

```

Feature Scaling

- **Fault:** The numerical features in the dataset, such as 'Cost_of_the_Product', 'Customer_rating', and 'Weight_in_gms', have different scales and varying units. Models like Logistic Regression and Neural Networks are sensitive to feature scales. For these models, features with larger-scale can dominating the learning process by appearing to be more important and degrading model performance.
- **Solution:** To standardize the range of all numerical features, Standard Scaling was applied. This method transforms the data to have a mean of 0 and a standard deviation of 1. Scaling was performed after the data was split into training and testing sets to prevent data leakage from the test set into the training process. We scaled based on the training data and fitted the Test data on the same scaling.

3.5 Class Imbalance Handling

As from the previous analysis, we found out that the dataset is not balanced. Although it is still somewhat balanced and won't affect the models performance by a significant margin but as our dataset nature makes it hard enough accurately predict, we will balance the data to give our models the best chance. To balance the data, we could use either oversampling or undersampling for our dataset to balance the 0 and 1 outcomes. Although oversampling is the norm, we went for undersampling to provide the models precise and raw data, not algorithmically generated data that can hamper the actual performance on real life data. We use **RandomUnderSampler** to randomly drop rows of 1's to make it equal to that of 0's.

```
1 # Handle class imbalance using RandomUnderSampler
2 from imblearn.under_sampling import RandomUnderSampler
3
4 rus = RandomUnderSampler(random_state=42)
5
6 print("Shape of training data before SMOTE:", X_train_processed.shape)
7 print("Class distribution before SMOTE:\n", y_train.value_counts())
8
9 X_train_resampled, y_train_resampled = rus.fit_resample(X_train_processed, y_train)
10
11 print("\nShape of training data after SMOTE:", X_train_resampled.shape)
12 print("Class distribution after SMOTE:\n", y_train_resampled.value_counts())
13
```

Shape of training data before SMOTE: (7699, 17)

Class distribution before SMOTE:

Reached.on.Time_Y.N

1 4594

0 3105

Name: count, dtype: int64

Shape of training data after SMOTE: (6210, 17)

Class distribution after SMOTE:

Reached.on.Time_Y.N

0 3105

1 3105

Name: count, dtype: int64

5. Model Training & Testing

As derived from the above analysis, this is a classification problem. In this part, we will discuss in details about the machine learning models that were used for this project. We will split the discussion into two parts: Supervised Learning and Unsupervised Learning.

5.1 Supervised Learning

For the supervised learning part, we used four learning models to predict the on-time delivery of shipments based on insightful features. As mentioned earlier, 70% of the data was used to train the models, and 30% of the data was used to test the models. Now we will further discuss about the models separately.

5.1.1 Logistic Regression

Logistic regression is a classification learning algorithm. The name Logistic Regression comes from statistics, and the mathematical formulation of it is similar to that of linear regression. It can predict the probability of an instance belonging to a discrete class. In our case, I will predict whether a shipment will arrive on time or not.

For our given project, we used the LogisticRegression model from a free and open-source machine learning library for Python called Scikit-learn. This library is vastly popular and an industry standard for its included classification, regression, and clustering algorithms. Logistic regression is considered a linear model based on its learning behaviour.

We used **GridSearchCV** (Grid Search Cross-Validation) for hyperparameter Tuning. We have specified a set of hyperparameters for the logistic regression model and used GridSearchCV to find the best combination of parameters based on the performance yields. By creating a grid of all possible combinations of the given hyperparameters, this tool evaluates each combination using cross-validation (cv=5).

The “**C**” parameter controls the strength of the regularization to prevent the model from overfitting. GridSearchCV sets the C as 0.1 which results in a strong regularization. As for the solver, “**liblinear**” uses a method called Coordinate Descent to minimize the log-loss on the predictions generated by the sigmoid function. After some warnings from the model, we set the max_iter to 1000, so the model can converge.

After training the model with the balanced training data, we tested the model for Accuracy, Precision, Recall, and F1-Score.


```

--- Classification Report for Logistic Regression ---
              precision    recall  f1-score   support

    0           0.55         0.78         0.64         1331
    1           0.79         0.56         0.66         1969

 accuracy          0.65         0.65         0.65         3300
 macro avg         0.67         0.67         0.65         3300
weighted avg         0.69         0.65         0.65         3300

```

5.1.2 Decision Tree

An acyclic graph that can be used to make decisions is a Decision Tree. We examine a specific feature vector in each branching node and reach the leaf node to get our desired decision. As a decision tree has a higher tendency to overfit, we apply appropriate parameters to control this. Like before, we use GridSearchCV with the DecisionTreeClassifier model from Scikit-learn. After 120 fits of GridSearchCV, we find that “**entropy**”, which uses the concept of entropy from information theory, is the best criterion for this classification problem. In order to control overfitting, a max_depth of 5 was selected, which creates a shallow tree with few decision rules. The minimum number of samples required to be in a node for it to be considered for splitting was set to 2, which may lead to overfitting, but combined with a max_depth of 5, it should be a balanced system. We also use cross-validation (cv=5) on this model.

```

--- Classification Report for Decision Tree ---
              precision    recall  f1-score   support

    0           0.56         0.97         0.71         1331
    1           0.96         0.48         0.64         1969

 accuracy          0.68         0.68         0.68         3300
 macro avg         0.76         0.73         0.68         3300
weighted avg         0.80         0.68         0.67         3300

```

The high recall of 0.97 for predicting 0 (Delayed) is a highlighting factor of the decision tree model.

5.1.3 Neural Network

Although the mathematical concepts are similar to those of Logistic regression, Neural Network works as a nested function. We used Keras, which runs on top of Tensorflow, to build our Neural Network model in a simpler way. We used Sequential to stack layers in a linear sequence. Then, we implemented Dense layers as the workhorse of our model. To regularize our model and prevent overfitting, we used Dropout.

The first hidden layer has 64 neurons in it. We used ReLU (Rectified Linear Unit) as our activation function. This helps the model to learn much more complex patterns by introducing non-linearity. The second hidden layer has 32 neurons and the same activation function. Right after that, we use a Dropout layer (regularization) to drop or temporarily deactivate 20% of the neurons from the previous layer so any single neuron don't become too specialized on a particular feature (overfitting). Then we use another hidden layer with 16 neurons and the same activation function. Finally, our output layer predicts one of two classes. We use the sigmoid activation function to turn it into a value between 0 and 1, which works as a probability.

The compile method configures the learning process of the model. As for the optimizer, we use the Adam (Adaptive Moment Estimation) algorithm. It adapts the Learning Rate for each weight in the network during training. As the problem we are dealing with is a binary classification problem, we use Binary Cross- Entropy as our loss function. It quantifies the difference between the predicted probabilities and the true labels from the output layer.

We use 25 epochs to loop through the dataset 25 times for adjusting the weights. For efficiency and a stable gradient estimation, we use the batch size of 32 for our model.

```
--- Classification Report for Neural Network ---
              precision    recall  f1-score   support

     0           0.55         0.97         0.70         1331
     1           0.96         0.45         0.61         1969

 accuracy                   0.66         3300
 macro avg           0.75         0.71         0.66         3300
 weighted avg        0.79         0.66         0.65         3300
```

5.1.4 K Nearest Neighbour

This is a supervised machine learning algorithm, which is used for classification and regression. This algorithm is also known as a lazy learner algorithm, as it does not learn from the training data; rather, it only works on the data it stores and computes the value at the time of classification. k values should be odd numbers, as it helps to avoid ties. GridSearchCV is used for hyperparameter tuning so that the model performs well. Param_grid is selected from 3 to 11 as it is best practice so that there is no underfit or overfit. Cross-validation is fivefold here to split the data; scoring is f1 to find out precision and recall. Euclidean distance is used to find the distance from the target point.

	POSITIVE	NEGATIVE		
POSITIVE	TP	FN	$Precision = \frac{TP}{TP + FP}$	$Recall = \frac{TP}{TP + FN}$
NEGATIVE	FP	TN	$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$	$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$

--- Classification Report for KNN ---				
	precision	recall	f1-score	support
0	0.55	0.73	0.63	1331
1	0.77	0.60	0.67	1969
accuracy			0.65	3300
macro avg	0.66	0.66	0.65	3300
weighted avg	0.68	0.65	0.65	3300

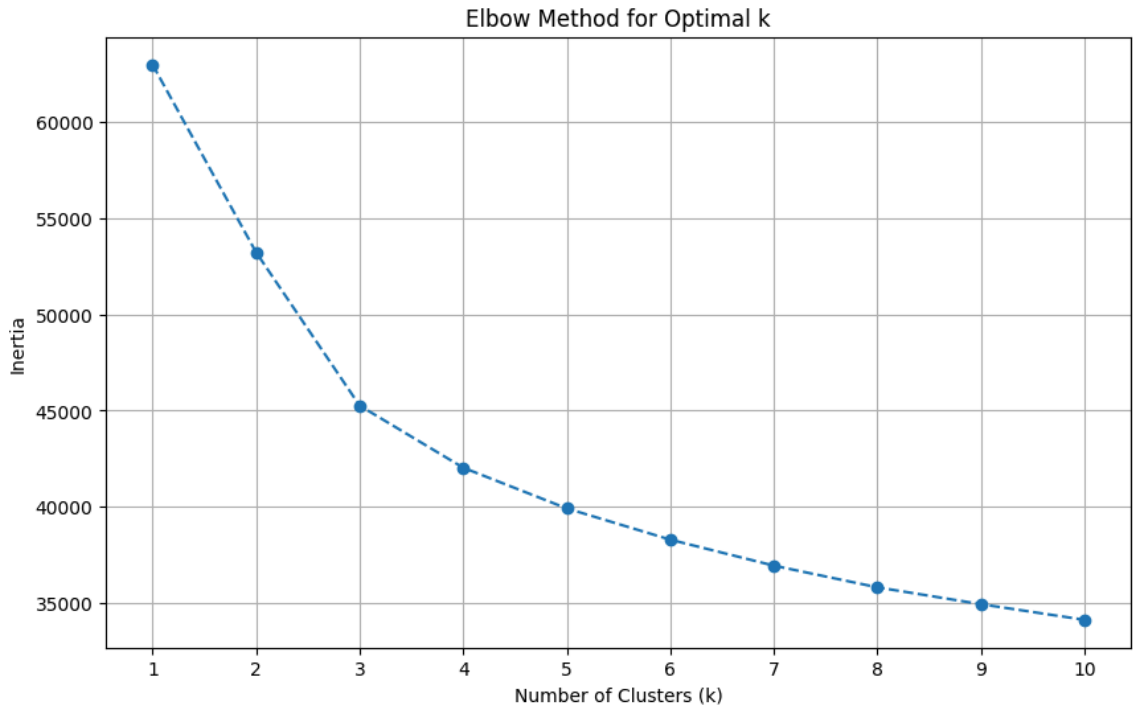
5.2 Unsupervised Learning

This is a machine learning approach where unlabeled data is given, and the algorithm learns from there and finds out the patterns and relationships without predefined output.

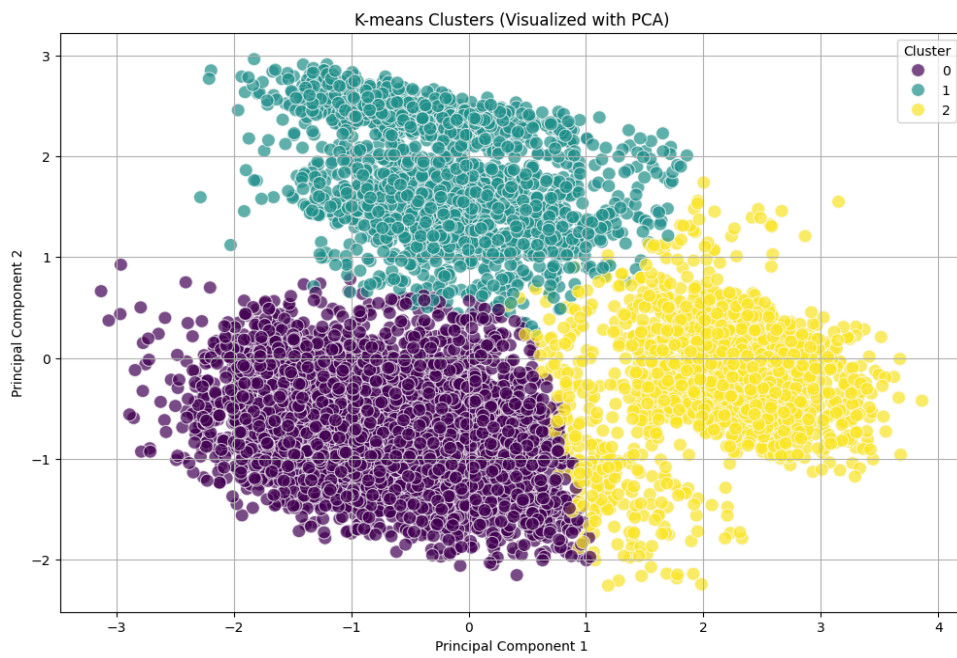
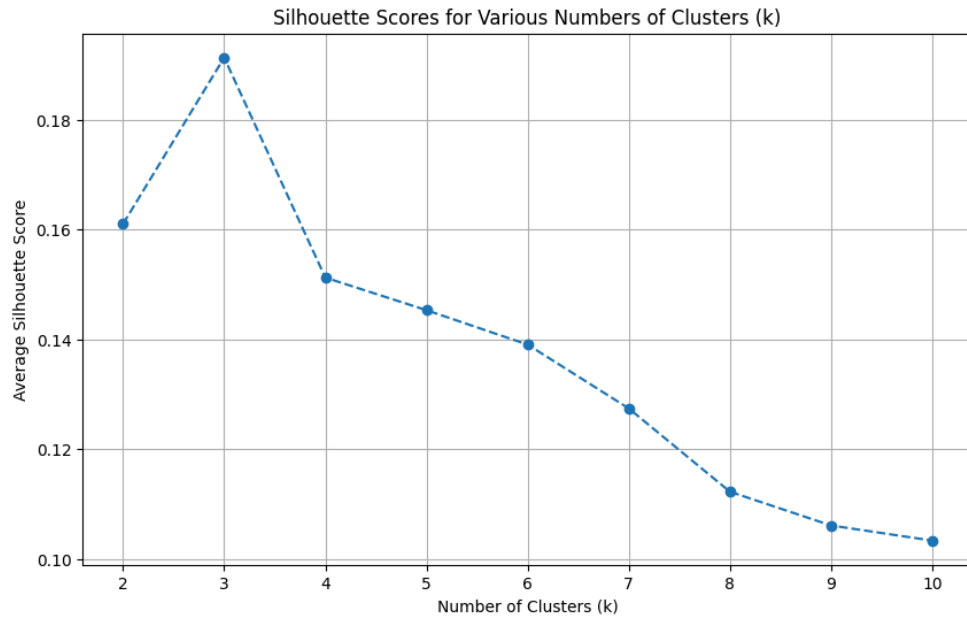
5.2.1 K Means

At first in this method, k cluster centroids will be initialized randomly. Then the algorithm will compute the distance of every point from the centroid and cluster accordingly. Then adjust the centroids so that they become the center of gravity for each

cluster. Then recluster the distance. Again, adjust the centroid until the new centroid and old centroid give the same value. Distance is calculated using the Euclidean method. To select the value of k , there is the Elbow Method, and in the Elbow Method, according to the cluster, the SSE is calculated, and their sum, which is the inertia. On the x-axis there is a k value, and the y-axis is holding the inertia, and where the curve turns, like the elbow, that is the best value of k .



Here, the k value is 3. Another is the Silhouette score for clustering. It is divided into two: one is cohesion and the other is separation. Suppose there are two clusters, cluster A and cluster B. Cohesion $a(i)$ mean of all the distances in the same cluster. Distance is calculated using Euclidean distance. $b(i)$ is for separation from each point in different clusters, and take the mean of it, suppose A_1, A_2 , and A_3 . Then the minimum of all the averages will be taken. The formula for the silhouette coefficient is $b-a/\max(b,a)$, and b should be large to get a good score. The coefficient should always be $[-1,1]$. For the value of 3 we get the maximum silhouette score and optimal k . PCA (principal component analysis) is used for reducing dimension.

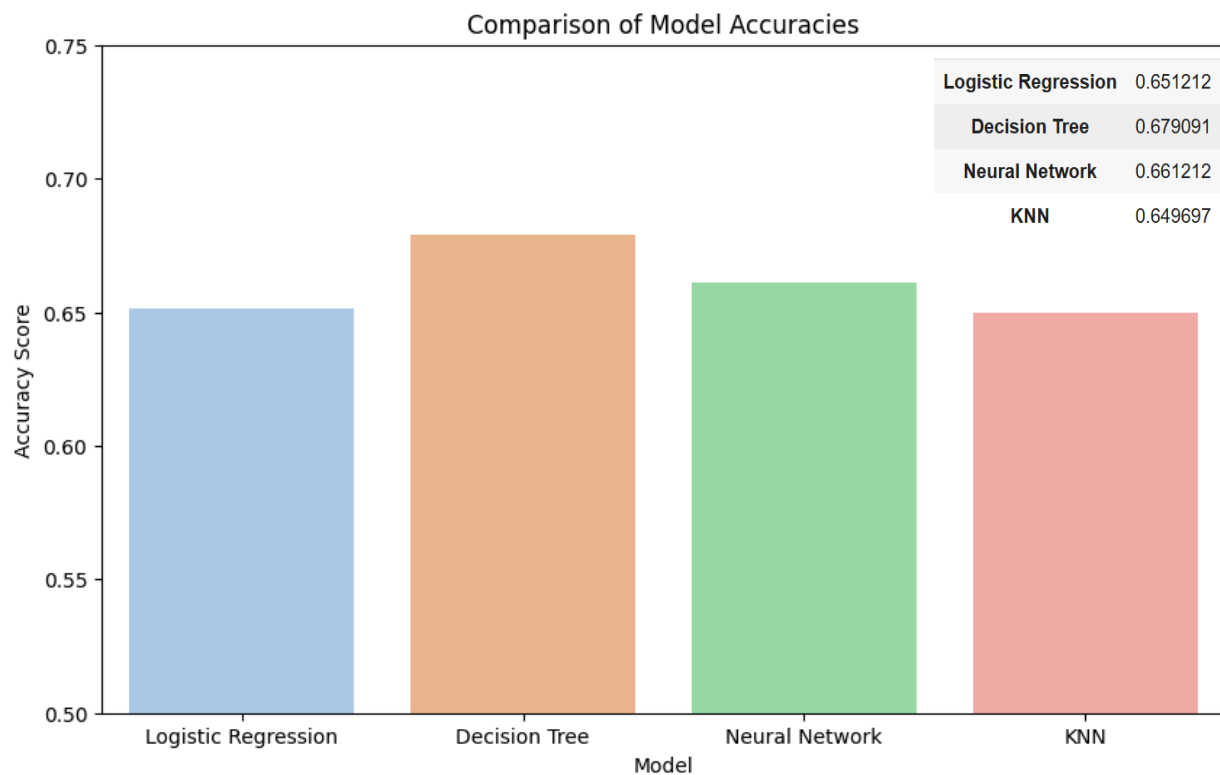


6. Comparative Analysis & Model Selection

In this section, we will compare the performance of our four supervised learning models based on some key classification metrics. Then, we will select the model that provides the most significant information with the best accuracy.

6.1 Comparison of Accuracy

From the provided bar chart, we can visualize the prediction accuracy of each model. We can get a high level yet clear overview and determine which model performed best on our dataset, overall. From the figure, we can see that the **Decision Tree** model have the highest accuracy among others. The **Decision Tree** model has an accuracy of 67%, while **Logistic Regression** (65%), **Neural Network** (66%), and **KNN** (64%) have lower accuracy compared to that.



6.2 Precision, Recall, and Confusion Matrices

While accuracy signifies the general measure of performance for each model, precision and recall gives us more in depth knowledge about the working process of the models, which is key to selecting the appropriate model for this particular problem.

6.2.1 Precision

Precision is the ratio of correct positive predictions to the overall number of positive predictions.

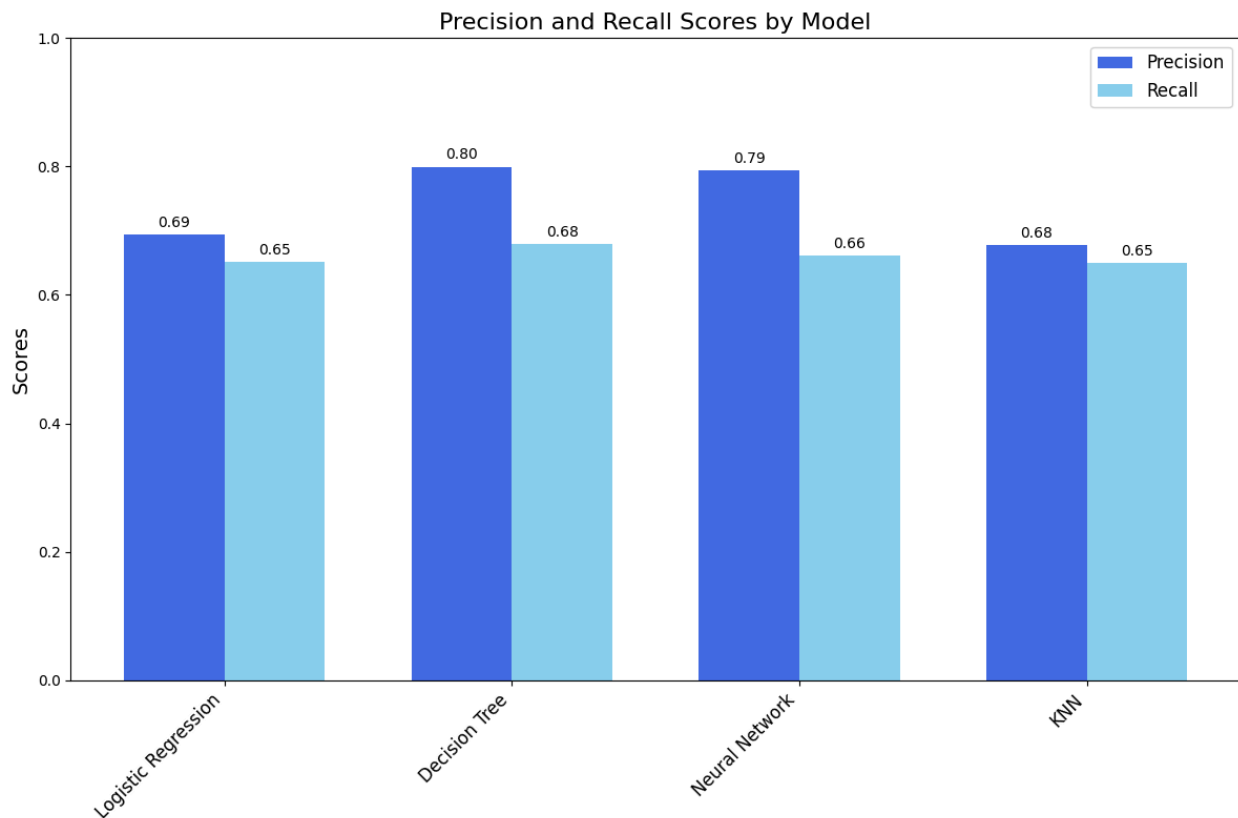
$$Precision = \frac{TP}{TP + FP}$$

6.2.2 Recall

Recall is the ratio of correct positive predictions to the overall number of positive examples in the dataset.

$$Recall = \frac{TP}{TP + FN}$$

Visualization of Precision and Recall (Weighted Average) for each model:



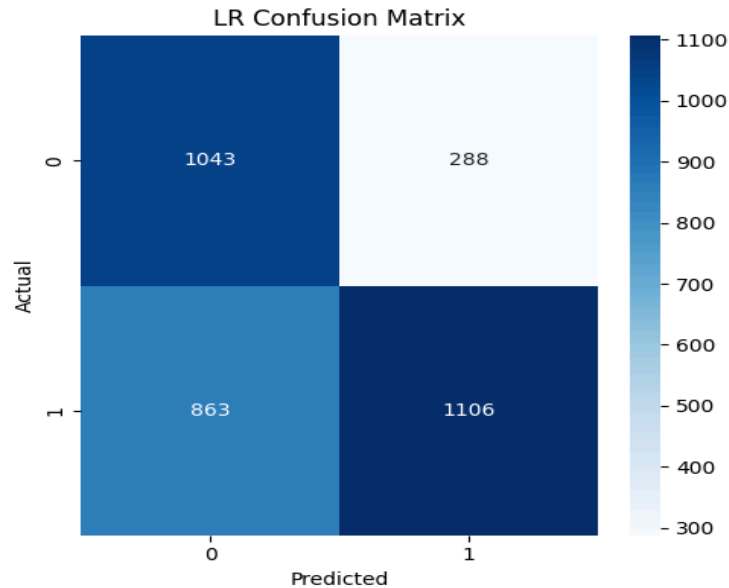
Note: All the models show higher precision than recall. **Decision Tree** has the highest precision and recall among all the others.

6.2.3 Confusion Matrices

A table that summarizes how successful the classification model is at predicting examples belonging to various classes is called the **Confusion Matrix**. The X axis of the confusion

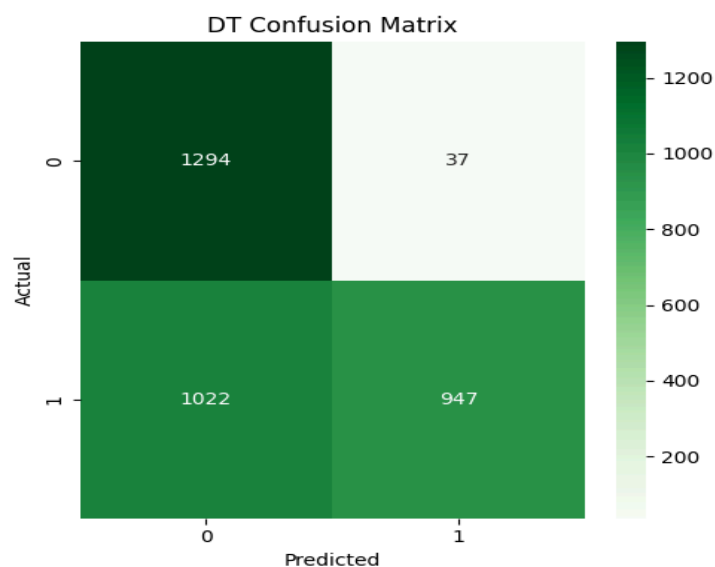
matrix is the label that the model predicted, and the Y axis is the actual label. For the binary classification problem of our, there are two classes, Delayed (0) and On-time (1).

- Logistic Regression:



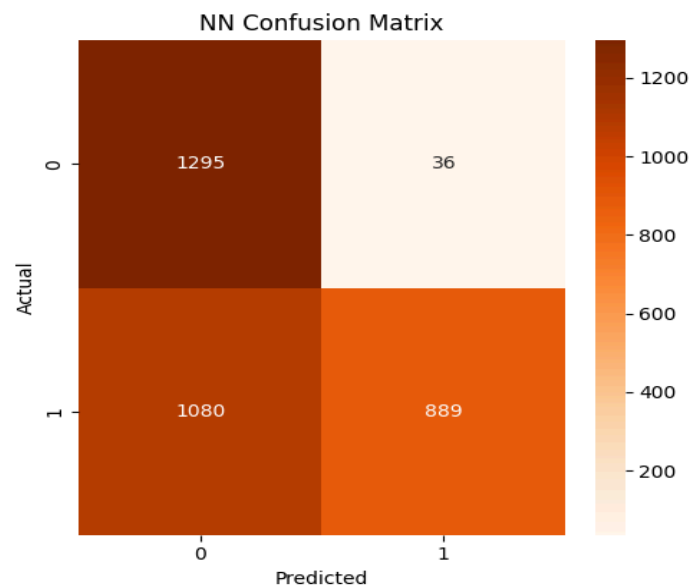
The confusion matrix of Logistic Regression shows, the model can predict shipments that will be **Delayed** better than the shipments that will be **On-time**. Only 288 shipments that were Delayed was predicted as On-time, on the other hand, a staggering 863 shipments that were On-time was predicted as Delayed. The confusion matrix displays that there is a high probability that the Logistic Regression model will flag a On-time shipment as Delayed.

- Decision Tree:



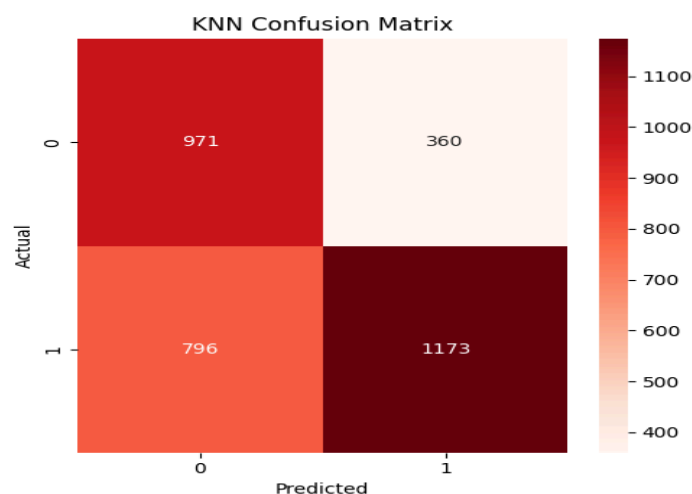
The Decision Tree model can predict Delayed shipments with a great accuracy with one of the lowest amount of False Positives (FP). Only 37 shipments were flagged On-time incorrectly which is a plus point for this model. This accuracy came with a cost, as 1022 shipments were flagged as Delayed even though they were actually On-time. A key information to point out is that, the number of False Negatives is higher than the number of True Positives. This means, the model is more likely to fail to determine a shipment On-time than succeed.

- Neural Network:



The Neural Network with 0.6 as the prediction threshold generates the lowest amount of False Positives (only 36). Additionally, the False Negatives are even higher than the Decision Tree. The difference between False Negatives and True Positives is even larger for the Neural Network model. This displays a significant lacking of the model for accurately predicting On-time shipment.

- K-Nearest Neighbors:



Finally, The KNN model shows the lowest amount of False Negatives (796), although still significantly large. Among other models used, **KNN** can most accurately predict a On-time shipment as On-time. The difference between KNN and Logistic regression is minimal but significant as KNN is more likely to fail to predict a Delayed shipment than Logistic Regression.

6.3 AUC Score and ROC Curve

The model's ability to distinguish between classes can be evaluated by the Area Under the Curve (AUC) and the Receiver Operating Characteristic (ROC) curve.

- **AUC:** The AUC score measures a model's ability to distinguish between the positive and negative classes across all possible decision thresholds. If a model gives a higher probability score to a positive instance than to a negative instance, the AUC score for that model will be higher.

Logistic regression = 0.72

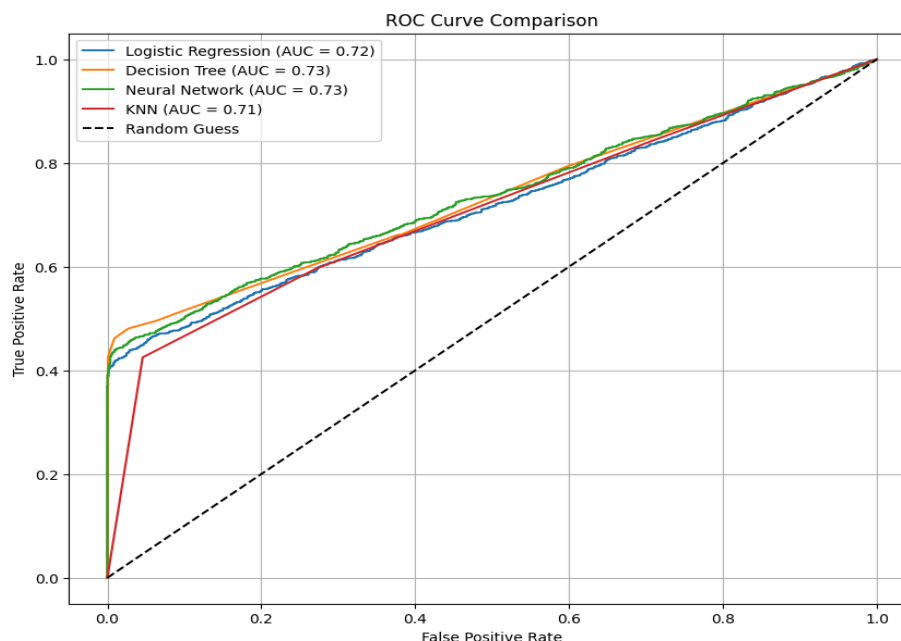
Decision Tree = 0.73

Neural Network = 0.73

K-Nearest Neighbors = 0.71

Here, the Decision Tree and Neural Network are slightly better at separating the On-time and Delayed classes than the KNN and Logistic Regression models. AUC score measures the potential but not the actual performance.

- **ROC Curve:** This graph shows how well a classification model can distinguish between two classes. The X axis represents False Positive Rate and the Y axis represents True Positive Rate.



6.4 Model Selection

For the given problem, Recall is the key metric to decide which model is best suited for the task of predicting On-time and Delayed shipments for a shipping company. Recall answers the question, How many actual positives did the model correctly predict? Recall for 0 measures the ability of a model to predict correctly the shipments that will actually be late. On the other hand, Recall for 1 measures the models ability to predict all the shipments that will actually arrive On-time correctly. A high recall is essential for choosing a model.

For a shipping company, the recall for 0 (delayed) is very important as flagging shipments that might be Delayed can significantly improve customer satisfaction and improve the overall quality of the business. Keeping that in mind, Decision Tree and Neural Network model with almost perfect score for 0 recall (0.97) looks like the obvious choice. This choice aligns with the necessity of predicting shipments that might get delayed.

Unfortunately, with a near perfect 0 recall, the Decision Tree and Neural Network models are extremely biased towards flagging a shipment Delayed. The 1 recall for both the models are very poor (48% and 45% respectively). This means that, system built on these models will flag more than 50% of the shipments Delayed, which will actually arrive on time.

To balance this problem, we have 2 other options, Logistic Regression and KNN model. The recall for 0 is 78% which is higher than 73% of KNN model. This implies that the logistic regression model will be better at flagging Delayed shipments. On the other hand, the recall for 1 is 56% for the Logistic model, which is less then that of KNN model (60%). To build a system using any one of these models, KNN is the most balance model that can somewhat accurately predict Delayed shipments along with less wrong predictions for On-time shipments. So, we can choose the KNN model this problem as it will be able to predict the Delayed shipments somewhat reliably along with flagging unnecessary Delays for shipments On-time.

7. Conclusion

This project demonstrates a machine learning pipeline for a real-world classification problem. In this project, we try to predict the on-time or delayed shipments of an e-commerce company. The accuracy of the models used is not very high, although better than random guessing. From the results, we can confirm our assumption that predicting the timely delivery of a shipment depends on more factors than those provided in the dataset. Human Error, Political Situation, Environmental Causes, etc., can also be considered as key features. The nature of this dataset allows lower accuracy for classification models than other classification friendly datasets.

Decision Tree and Neural Network models provided somewhat biased performance with high recall for one type of class sacrificing the other one. On the other hand, Logistic Regression and K-Nearest Neighbors also provided better recall for 0 (Delayed) but the results were more balanced than the previous two. All of the models performed better to identify Delayed shipments. The precision score were consistently higher than the recall score.

The models performed satisfactorily given the dataset. The results are not the highest, and the prediction confidence is somewhat typical. The features were not particularly correlated with each other or the target feature. Hypertuning the parameters for each model increased the model accuracy by a significant margin. Further improvements came through rigorous preprocessing of data. Overall, the result of the models in this project is on par with the results other researchers are getting.

The most difficult part was to increase the accuracy of the models. We had to try every combination of preprocessing steps to identify the key issue and later settle for what we got. Preprocessing the dataset to best fit the models was the hardest part of this project.